



3-Month C++ DSA Roadmap (Mon-Fri)

WEEK 1: Basics + Logic Building (Lectures 1-10)

Goal: Build a strong programming foundation in C++

- **Mon:** Lecture 1 & 2 (Intro to Programming, Flowcharts, First C++ Program & Data Types)
- **Tue:** Lecture 3 & 4 (Conditionals `if-else`, `while` loops & Patterns Part-1)
- **Wed:** Lecture 5 & 6 (Bitwise Operators, `for` loops & Binary/Decimal Number System)
- **Thu:** Lecture 7 & 8 (LeetCode Problem Solving Session, Switch Case & Functions)
- **Fri:** Lecture 9 & 10 (Introduction to Arrays & LeetCode Array Problems)

WEEK 2: Complexity + Binary Search Intro (Lectures 11-15)

Goal: Learn how to analyze code & crack Binary Search logic

- **Mon:** Lecture 11 (Time & Space Complexity Deep Dive)
- **Tue:** Lecture 12 (Binary Search Concept & Implementation)
- **Wed:** Lecture 13 (Binary Search Interview Questions - First/Last Occurrence)
- **Thu:** Lecture 14 (Binary Search Pivot Element & Square Root Problems)
- **Fri:** Lecture 15 (Advanced BS Problems: Book Allocation & Painter's Partition)

WEEK 3: Sorting + STL + Strings (Lectures 16-24)

Goal: Learn sorting algorithms and standard C++ tools

- **Mon:** Lecture 16 & 17 (Selection Sort & Bubble Sort Implementation)
- **Tue:** Lecture 18 & 19 (Insertion Sort & C++ Standard Template Library - STL)
- **Wed:** Lecture 20 & 21 (Array LeetCode Questions & Solving String Problems)
- **Thu:** Lecture 22 & 23 (Character Arrays, Strings & 2D Arrays Intro)
- **Fri:** Lecture 24 (Basic Mathematics for Coding / Sieve of Eratosthenes)

WEEK 4: Pointers + Memory Management (Lectures 25-30)

Goal: Master the core of C++: Memory allocation

- **Mon:** Lecture 25 (Pointers in C++ Part-1: Address & Dereference)
- **Tue:** Lecture 26 (Pointers with Arrays & Functions)
- **Wed:** Lecture 27 (Double Pointers & Pointer Concept MCQ Check)
- **Thu:** Lecture 28 (Reference Variables & Static vs Dynamic Memory Allocation)
- **Fri:** Lecture 29 & 30 (Dynamic Allocation in 2D Arrays, Macros & Inline Functions)

WEEK 5: Recursion Basics (Lectures 31-34)

Goal: Understand how the function call stack works (Do not rush!)

- **Mon:** Lecture 31 (Recursion Intro: Base Case, Recurrence Relation, Factorial)
- **Tue:** Lecture 32 (Walking/Counting Problems & Say Digits using Recursion)
- **Wed:** Lecture 33 (Recursion with Arrays: Is Sorted & Linear Search)
- **Thu:** Lecture 34 (Binary Search using Recursion)
- **Fri:** String Recursion Practice (Reverse String, Palindrome, and Power Check)

WEEK 6: Advanced Recursion + Backtracking (Lectures 35-40)

Goal: Master sorting with recursion and classic recursion problems

- **Mon:** Lecture 35 (Merge Sort using Recursion - Very Important)
- **Tue:** Lecture 36 (Quick Sort using Recursion)
- **Wed:** Lecture 37 (Subsets & Subsequences of Strings)
- **Thu:** Lecture 38 & 39 (Phone Keypad Problem & Permutations of a String)
- **Fri:** Lecture 40 (Rat in a Maze Problem - Your first Backtracking concept)

WEEK 7: Object-Oriented Programming (OOPs) (Lectures 41-43)

Goal: Learn OOPs pillars for interviews

- **Mon:** Lecture 41 (OOPs Concepts Part-1: Classes, Objects, Access Modifiers)
- **Tue:** Lecture 42 (Encapsulation, Inheritance & Polymorphism)
- **Wed:** Lecture 43 (Abstraction & Advanced OOPs Concept Checklist)
- **Thu:** Rewatch and practice OOPs concepts on paper
- **Fri:** Create a small, clean C++ project using OOPs (e.g., Student Management System)

WEEK 8: Linked List Core (Lectures 44-47)

Goal: Master the most asked linear data structure

- **Mon:** Lecture 44 (Singly Linked List Implementation: Insertion & Deletion)
- **Tue:** Lecture 45 (Doubly Linked List & Circular Linked List Implementation)
- **Wed:** Lecture 46 (Reverse a Linked List & Find Middle Node)
- **Thu:** Lecture 47 (Reverse LL in K-Groups & Check if Loop/Cycle Exists)
- **Fri:** Loop Removal Practice (Floyd's Cycle Detection Algorithm dry-run)

WEEK 9: Linked List Advanced Problems (Lectures 48-53)

Goal: Solve top interview problems on Linked Lists

- **Mon:** Lecture 48 (Remove Duplicates from Sorted & Unsorted Linked List)
- **Tue:** Lecture 49 (Sort 0s, 1s, 2s in a LL & Merge 2 Sorted LL)
- **Wed:** Lecture 50 & 51 (Check Palindrome LL & Add 2 Numbers represented by LL)
- **Thu:** Lecture 52 (Clone a Linked List with Random Pointers)
- **Fri:** Lecture 53 (Merge Sort in Linked List + LL Revision)

WEEK 10: Stacks & Queues (Lectures 54-61)

Goal: Learn LIFO and FIFO operations and their applications

- **Mon:** Lecture 54 & 55 (Stack Implementation using Arrays/LL & Two Stacks in an Array)
- **Tue:** Lecture 56 & 57 (Reverse a String / Delete Middle of Stack & Valid Parentheses)
- **Wed:** Lecture 58 & 59 (Insert at Bottom, Reverse Stack & Sort a Stack)
- **Thu:** Lecture 60 & 61 (Redundant Brackets, Minimum Cost to Make String Valid)
- **Fri:** Stack Question Practice (Next Smaller Element logic)

WEEK 11: Queue Core + Binary Trees Intro (Lectures 62-64)

Goal: Finish Queues and step into Non-Linear Data Structures

- **Mon:** Lecture 62 (Queue Implementation, Circular Queue & Input/Output Restricted Deque)
- **Tue:** Lecture 63 (Queue Interview Questions: Reverse Queue, First Negative in K-window)
- **Wed:** Lecture 64 (Binary Trees Introduction & Level Order Traversal)
- **Thu:** Inorder, Preorder, and Postorder Traversals practice on paper
- **Fri:** Build Tree from Level Order Traversal code implementation

WEEK 12: Binary Trees Advanced & BST (Lectures 65-80)

 *Final Week Intensive: Push 5-6 hours/day to finish the core*

- **Mon:** Lecture 65 & 66 (Height, Diameter, and Balance check of a Binary Tree)
- **Tue:** Lecture 67 & 68 (Boundary, Vertical, Top, and Bottom View of a Tree)
- **Wed:** Lecture 69 & 70 (Sum Tree, Longest Bloodline Path, Lowest Common Ancestor)
- **Thu:** Lecture 71, 72 & 73 (Construct Tree from Inorder/Preorder, Burning Tree Question)
- **Fri:** Lecture 74 to 80 (Binary Search Tree - BST Basics, Validation, Deletion, and Conversion)

Final Day Note:

You have successfully and safely completed the core C++ DSA syllabus in 90 days! Move on to Advanced Graphs and Dynamic Programming (DP) only after this base is crystal clear.